



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Learning to Control PDEs with Differentiable Predictive Control and Neural Operators

Ján Drgoňa

Associate Professor

Civil and Systems Engineering Department

Electrical and Computer Engineering Department (secondary)

The Ralph O'Connor Sustainable Energy Institute (ROSEI)

Data Science and AI Institute (DSAI)

State of the Art Control Methods

Model Predictive Control

95% of the industrial control applications are PID loops & expert driven rule-based control (RBC).
The rest is model predictive control (MPC).



Reinforcement Learning

Everywhere where you have reliable digital environment model, and you can use compute resources to solve the problem offline. Successful in GenAI, games, and robotics.



James B. Rawlings, David Q. Mayne, Moritz M. Diehl, Model Predictive Control: Theory and Design, Nob Hill Publishing, 2nd ed., 2017
Richard S. Sutton and Andrew G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2nd ed., 2018
Drgona J., et al. 2020. "All You Need to Know About Model Predictive Control for Buildings." Annual Reviews in Control, September 29, 2020
G. Dulac-Arnold, D. J. Mankowitz, and T. Hester, "Challenges of real-world reinforcement learning," 2019.

Model Predictive Control (MPC)

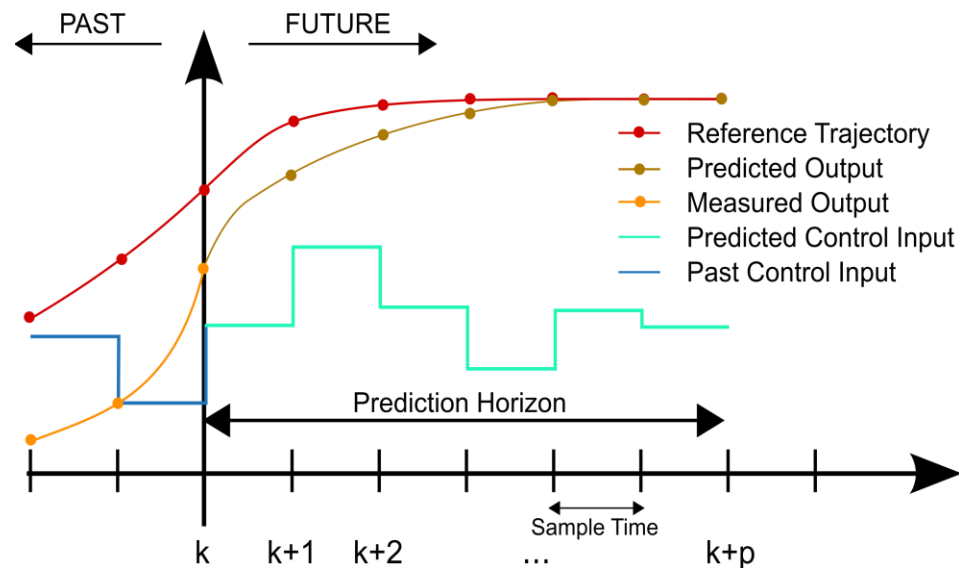
MPC uses a **receding horizon** approach to compute the optimal control input at each time step by solving a **finite-horizon** optimal control problem in real-time via **online optimization** using numerical methods.

Finite-horizon optimal control problem formulation:

$$\begin{aligned} \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \quad & \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + p_N(\mathbf{x}_N) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \quad k \in \mathbb{N}_0^{N-1} \\ & h(\mathbf{x}_k) \leq 0 \\ & g(\mathbf{u}_k) \leq 0 \\ & \mathbf{x}_0 = \mathbf{x}(t) \end{aligned}$$

At each time step:

1. Measure the current state x_{current} .
2. Solve the finite-horizon optimization problem for N future steps.
3. Apply only the first control input u_0^* .
4. Move to the next time step and repeat.



Model Predictive Control (MPC)

MPC uses a **receding horizon** approach to compute the optimal control input at each time step by solving a **finite-horizon** optimal control problem in real-time via **online optimization** using numerical methods.

Finite-horizon optimal control problem formulation:

$$\begin{aligned} \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \quad & \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + p_N(\mathbf{x}_N) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \quad k \in \mathbb{N}_0^{N-1} \\ & h(\mathbf{x}_k) \leq 0 \\ & g(\mathbf{u}_k) \leq 0 \\ & \mathbf{x}_0 = \mathbf{x}(t) \end{aligned}$$

At each time step:

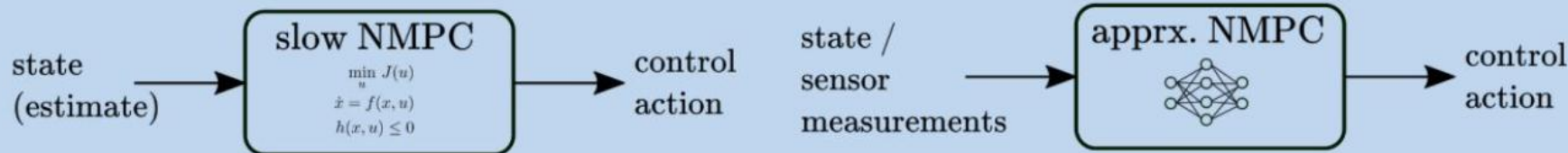
1. Measure the current state $\mathbf{x}_{\text{current}}$.
2. Solve the finite-horizon optimization problem for N future steps.
3. Apply only the first control input $\mathbf{u}_0^*$.
4. Move to the next time step and repeat.

MPC software tools:



Challenges: Computationally expensive online.

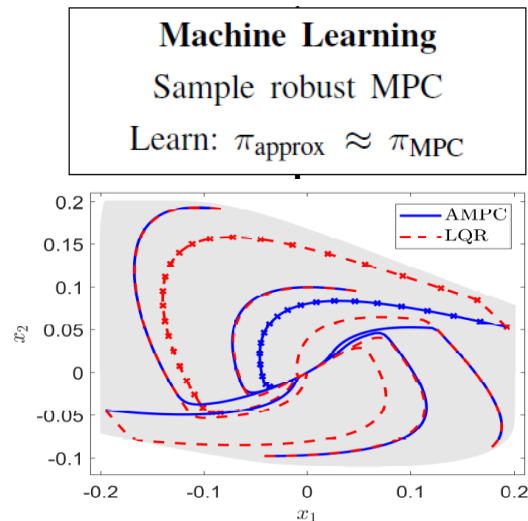
Approximate MPC



Step 1: solve set of MPC problems to generate labeled training data

$$\min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + p_N(\mathbf{x}_N)$$
$$\text{s.t. } \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \quad k \in \mathbb{N}_0^{N-1}$$
$$h(\mathbf{x}_k) \leq 0$$
$$g(\mathbf{u}_k) \leq 0$$
$$\mathbf{x}_0 = \mathbf{x}(t)$$

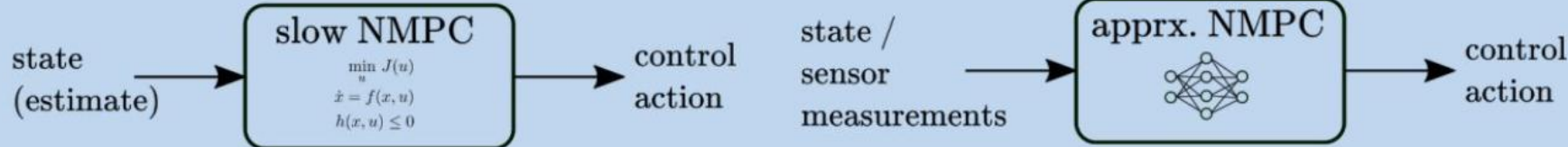
Step 2: supervised imitation learning to learn approximate MPC policy



M. Hertneck, et al., "Learning an Approximate Model Predictive Controller With Guarantees," in IEEE Control Systems Letters, 2018

B. Karg and S. Lucia, "Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning," in IEEE Transactions on Cybernetics, 2020

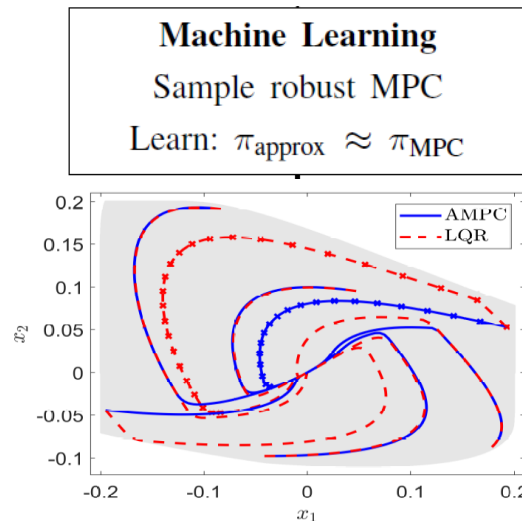
Approximate MPC



Step 1: solve set of MPC problems to generate labeled training data

$$\begin{aligned} \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \quad & \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + p_N(\mathbf{x}_N) \\ \text{s.t. } \quad & \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \quad k \in \mathbb{N}_0^{N-1} \\ & h(\mathbf{x}_k) \leq 0 \\ & g(\mathbf{u}_k) \leq 0 \\ & \mathbf{x}_0 = \mathbf{x}(t) \end{aligned}$$

Step 2: supervised imitation learning to learn approximate MPC policy



Challenges:

Expensive training data generation.

Knowledge of the model and constraints not utilized in learning.

M. Hertneck, et al., "Learning an Approximate Model Predictive Controller With Guarantees," in IEEE Control Systems Letters, 2018

B. Karg and S. Lucia, "Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning," in IEEE Transactions on Cybernetics, 2020

Reinforcement Learning (RL)

RL is a **data-driven decision-making framework** where an agent interacts with an environment to learn an optimal policy that maximizes rewards over **infinite-horizon**. Most modern **policy-gradient methods** (actor-critic, PPO, TRPO, DDPG, TD3, offline RL) **learn a critic** and use it to **update the actor (policy)**.

1, **Data collection** via sampling:

$$x_t \sim \text{environment}$$

$$u_t = \pi_\theta(x_t)$$

$$x_{t+1}, r_t \sim \mathcal{P}(\cdot \mid x_t, u_t)$$

$$(x_t, u_t, r_t, x_{t+1}) \in \mathcal{D}$$

2, **Critic Update:**

Q-function

Bellman target

Critic loss function

$$Q^\pi(x, u) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_k \mid x_0 = x, u_0 = u, \pi \right]$$

$$y_t = r_t + \gamma Q_{\phi'}(x_{t+1}, \pi_{\theta'}(x_{t+1}))$$

$$\mathcal{L}_{\text{critic}}(\phi) = (Q_\phi(x_t, u_t) - y_t)^2$$

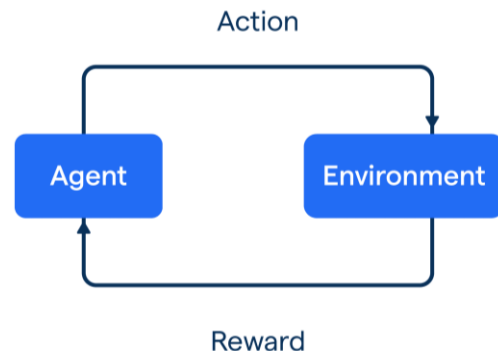
3, **Actor update** via deterministic policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\nabla_u Q_\phi(x, u) \Big|_{u=\pi_\theta(x)} \cdot \nabla_\theta \pi_\theta(x) \right]$$

4, **Gradient descent updates:**

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$



Reinforcement Learning (RL)

RL is a **data-driven decision-making framework** where an agent interacts with an environment to learn an optimal policy that maximizes rewards over **infinite-horizon**. Most modern **policy-gradient methods** (actor-critic, PPO, TRPO, DDPG, TD3, offline RL) **learn a critic** and use it to **update the actor (policy)**.

1, **Data collection** via sampling:

$$x_t \sim \text{environment}$$

$$u_t = \pi_\theta(x_t)$$

$$x_{t+1}, r_t \sim \mathcal{P}(\cdot \mid x_t, u_t)$$

$$(x_t, u_t, r_t, x_{t+1}) \in \mathcal{D}$$

2, **Critic Update:**

Q-function

Bellman target

Critic loss function

$$Q^\pi(x, u) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_k \mid x_0 = x, u_0 = u, \pi \right]$$

$$y_t = r_t + \gamma Q_{\phi'}(x_{t+1}, \pi_{\theta'}(x_{t+1}))$$

$$\mathcal{L}_{\text{critic}}(\phi) = (Q_\phi(x_t, u_t) - y_t)^2$$

3, **Actor update** via deterministic policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\nabla_u Q_\phi(x, u) \Big|_{u=\pi_\theta(x)} \cdot \nabla_\theta \pi_\theta(x) \right]$$

4, **Gradient descent updates:**

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$

RL software tools:

 PyTorch  TensorFlow



Challenges: Sample inefficient! Hard to obtain guarantees.

Model-Based Reinforcement Learning (MBRL)

MBRL aims to **improve sample efficiency** by **learning a predictive model** of the environment dynamics and using it to **simulate trajectories** for Q-function fitting and policy optimization. This framework is followed by methods such as MBPO, TD3-MB, MOPO, COMBO, DreamerV3, and Plan2Explore.

1, **Model rollouts** via sampling:

$$\hat{x}_0 = x \sim \mathcal{D}$$

$$\hat{u}_t \sim \pi_\theta(\cdot \mid \hat{x}_t)$$

$$\hat{x}_{t+1} = f_\psi(\hat{x}_t, \hat{u}_t)$$

$$\hat{r}_t = \hat{r}_\psi(\hat{x}_t, \hat{u}_t)$$

2, **Critic Update** using model rollouts:

Bellman target $y_t = \hat{r}_t + \gamma Q_{\phi'}(\hat{x}_{t+1}, \pi_{\theta'}(\hat{x}_{t+1}))$

Critic loss function $\mathcal{L}_{\text{critic}}(\phi) = (Q_\phi(\hat{x}_t, \hat{u}_t) - y_t)^2$

3, **Actor update** via deterministic policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\hat{x}_t \sim \mathcal{D}} \left[\nabla_u Q_\phi(\hat{x}_t, u) \Big|_{u=\pi_\theta(\hat{x}_t)} \cdot \nabla_\theta \pi_\theta(\hat{x}_t) \right]$$

4, **Gradient descent updates:**

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$

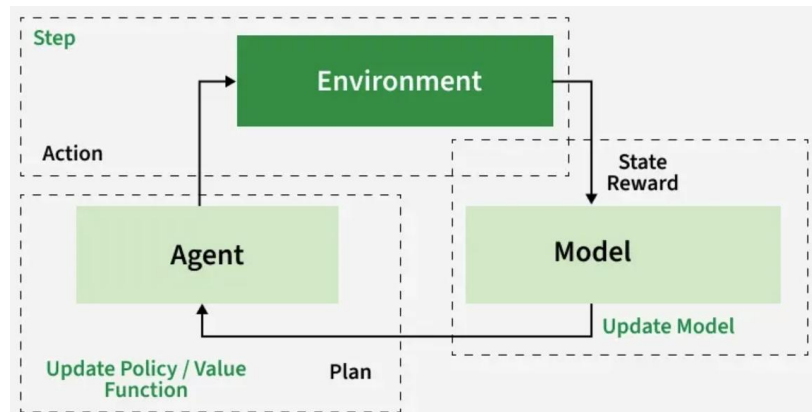


Image: www.geeksforgeeks.org/artificial-intelligence/model-based-reinforcement-learning-mbri-in-ai/

Model-Based Reinforcement Learning (MBRL)

MBRL aims to **improve sample efficiency** by **learning a predictive model** of the environment dynamics and using it to **simulate trajectories** for Q-function fitting and policy optimization. This framework is followed by methods such as MBPO, TD3-MB, MOPO, COMBO, DreamerV3, and Plan2Explore.

1, **Model rollouts** via sampling:

$$\hat{x}_0 = x \sim \mathcal{D}$$

$$\hat{u}_t \sim \pi_\theta(\cdot \mid \hat{x}_t)$$

$$\hat{x}_{t+1} = f_\psi(\hat{x}_t, \hat{u}_t)$$

$$\hat{r}_t = \hat{r}_\psi(\hat{x}_t, \hat{u}_t)$$

Why to learn the critic if we have accurate and differentiable model and parametric reward functions?

2, **Critic Update** using model rollouts:

Bellman target

$$y_t = \hat{r}_t + \gamma Q_{\phi'}(\hat{x}_{t+1}, \pi_{\theta'}(\hat{x}_{t+1}))$$

Critic loss function

$$\mathcal{L}_{\text{critic}}(\phi) = (Q_\phi(\hat{x}_t, \hat{u}_t) - y_t)^2$$

3, **Actor update** via deterministic policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\hat{x}_t \sim \mathcal{D}} \left[\nabla_u Q_\phi(\hat{x}_t, u) \Big|_{u=\pi_\theta(\hat{x}_t)} \cdot \nabla_\theta \pi_\theta(\hat{x}_t) \right]$$

4, **Gradient descent updates:**

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$

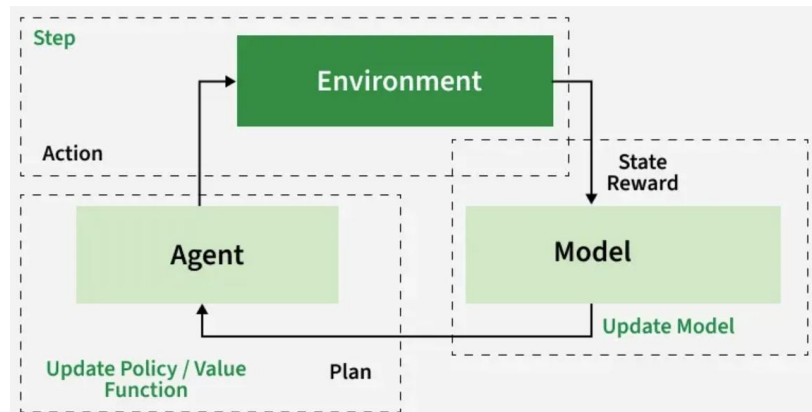
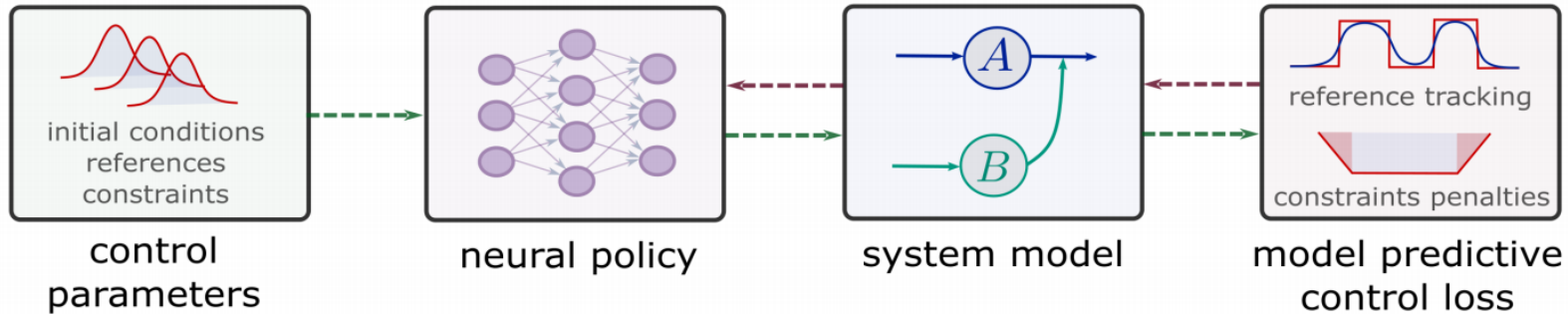
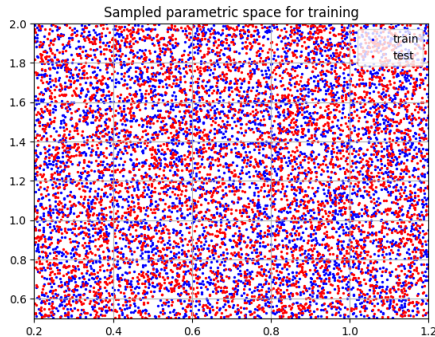


Image: www.geeksforgeeks.org/artificial-intelligence/model-based-reinforcement-learning-mbri-in-ai/

Differentiable Predictive Control (DPC)



Dataset: collocation points in the control parametric space.



Architecture: differentiable model with neural network control policy.

$$x_{k+1} = \text{ODESolve}(f(x_k, u_k))$$

$$u_k = NN_{\theta}(x_k, \xi_k)$$

$$g(x_k, u_k, \xi_k) \leq 0$$

$$x_0 \sim \mathcal{P}_{x_0}$$

$$\xi_k \sim \mathcal{P}_{\xi}$$

Loss function: reference tracking, constraints and terminal penalties.

$$\ell_1 = \sum_{i=1}^m \sum_{k=1}^{N-1} Q_x \|x_k^i - r_k^i\|_2^2$$

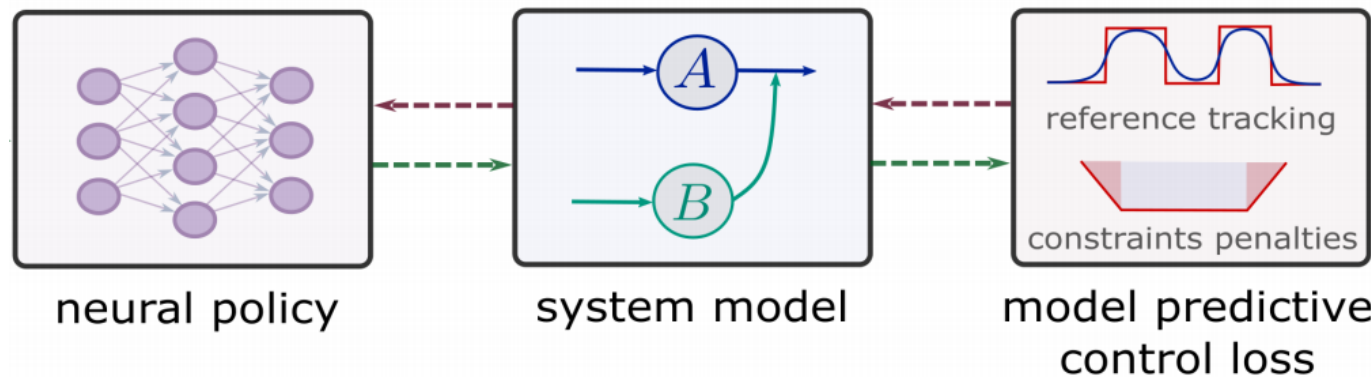
$$\ell_2 = \sum_{i=1}^m \sum_{k=1}^{N-1} Q_g \|\text{RELU}(g(x_k^i, u_k^i, \xi_k^i))\|_2^2$$

$$\ell_{L2C} = \ell_1 + \ell_2$$

J. Dragoña, A. Tuor and D. Vrabie, "Learning Constrained Parametric Differentiable Predictive Control Policies With Guarantees," in IEEE Transactions on Systems, Man, and Cybernetics: Systems, 2024

Ján Dragoña, et al, Differentiable predictive control: Deep learning alternative to explicit model predictive control for unknown nonlinear systems, Journal of Process Control, 2022

Differentiable Closed-Loop System



Forward pass. For a single scenario, the closed-loop recursion and loss are

$$\begin{aligned}u_k &= \pi_{\theta}(x_k; \xi_k), \\x_{k+1} &= f(x_k, u_k; \xi_k), \\ \ell_k &= \ell(x_k, u_k; \xi_k) + p_x(g(x_k; \xi_k)) + p_u(h(u_k; \xi_k)), \\ \mathcal{L} &= \frac{1}{N} \sum_{k=0}^{N-1} \ell_k + \ell_N(x_N; \xi_N).\end{aligned}$$

Differentiable Closed-Loop System

Backward pass (sensitivities). Let

$$A_k := \frac{\partial f}{\partial x}, \quad B_k := \frac{\partial f}{\partial u}, \quad P_k := \frac{\partial \pi_\theta}{\partial x}, \quad G_k := \frac{\partial \pi_\theta}{\partial \theta} \quad (\text{all evaluated at } (x_k, u_k; \xi_k)).$$

Define $s_N := \frac{\partial \ell_N}{\partial x}(x_N; \xi_N)$ and stage gradients

$$\ell_{x,k} := \frac{\partial \ell}{\partial x} + \frac{\partial p_x}{\partial x}, \quad \ell_{u,k} := \frac{\partial \ell}{\partial u} + \frac{\partial p_u}{\partial u}.$$

Then the reverse recursion for $k = N - 1, \dots, 0$ is

$$s_k = \ell_{x,k} + A_k^\top s_{k+1} + P_k^\top (\ell_{u,k} + B_k^\top s_{k+1}),$$

and the gradient w.r.t. the policy parameters is

$$\nabla_\theta \mathcal{L} = \sum_{k=0}^{N-1} G_k^\top (\ell_{u,k} + B_k^\top s_{k+1}).$$

Differentiable Closed-Loop System

Backward pass (sensitivities). Let

$$A_k := \frac{\partial f}{\partial x}, \quad B_k := \frac{\partial f}{\partial u}, \quad P_k := \frac{\partial \pi_\theta}{\partial x}, \quad G_k := \frac{\partial \pi_\theta}{\partial \theta} \quad (\text{all evaluated at } (x_k, u_k; \xi_k)).$$

Define $s_N := \frac{\partial \ell_N}{\partial x}(x_N; \xi_N)$ and stage gradients

$$\ell_{x,k} := \frac{\partial \ell}{\partial x} + \frac{\partial p_x}{\partial x}, \quad \ell_{u,k} := \frac{\partial \ell}{\partial u} + \frac{\partial p_u}{\partial u}.$$

Then the reverse recursion for $k = N - 1, \dots, 0$ is

$$s_k = \ell_{x,k} + A_k^\top s_{k+1} + P_k^\top (\ell_{u,k} + B_k^\top s_{k+1}),$$

and the gradient w.r.t. the policy parameters is

$$\nabla_\theta \mathcal{L} = \sum_{k=0}^{N-1} G_k^\top (\ell_{u,k} + B_k^\top s_{k+1}).$$

Backward pass in DPC are discrete adjoint sensitivities leading to Pontryagin costates in continuous time.

DPC Policy Optimization Algorithm

Algorithm 5: Self-Supervised DPC Policy Optimization

Input: Horizon N , batch size B ; initial-state distribution $\mathcal{X}_0$, scenario distribution $\mathcal{D}$;
dynamics $x_{k+1} = f(x_k, u_k; \xi_k)$; policy class $u_k = \pi_\theta(x_k; \xi_k)$; costs ℓ, ℓ_N ; penalties
 p_x, p_u with weights Q_x, Q_u ; constraints g, h ; optimizer $\mathbb{O}$

Output: Trained explicit policy π_{θ^*}

```
1 while not converged do
2   Sample mini-batch  $\{(x_0^{(i)}, \xi^{(i)})\}_{i=1}^B \sim \mathcal{X}_0 \times \mathcal{D}$ ;
3   for  $i \leftarrow 1$  to  $B$  do
4     for  $k \leftarrow 0$  to  $N - 1$  do
5        $u_k^{(i)} \leftarrow \pi_\theta(x_k^{(i)}; \xi_k^{(i)})$ ;
6        $x_{k+1}^{(i)} \leftarrow f(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)})$ ;
7        $\ell_k^{(i)} \leftarrow \ell(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}) + Q_x p_x(g(x_k^{(i)}; \xi_k^{(i)})) + Q_u p_u(h(u_k^{(i)}; \xi_k^{(i)}))$ ;
8      $\mathcal{L}^{(i)} \leftarrow \frac{1}{N} \sum_{k=0}^{N-1} \ell_k^{(i)} + \ell_N(x_N^{(i)}; \xi_N^{(i)})$ ;
9    $\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^B \mathcal{L}^{(i)}$ ; // batch DPC loss
10  Compute  $\nabla_\theta \mathcal{L}$  (e.g., BPTT or adjoint sensitivity using (8.3)–(8.4));
11   $\theta \leftarrow \mathbb{O}(\theta, \nabla_\theta \mathcal{L})$ ; // e.g., Adam step
12 return  $\pi_\theta$ ;
```

DPC Policy Optimization Algorithm

Algorithm 5: Self-Supervised DPC Policy Optimization

Input: Horizon N , batch size B ; initial-state distribution $\mathcal{X}_0$, scenario distribution $\mathcal{D}$;
 dynamics $x_{k+1} = f(x_k, u_k; \xi_k)$; policy class $u_k = \pi_\theta(x_k; \xi_k)$; costs ℓ, ℓ_N ; penalties
 p_x, p_u with weights Q_x, Q_u ; constraints g, h ; optimizer $\mathbb{O}$

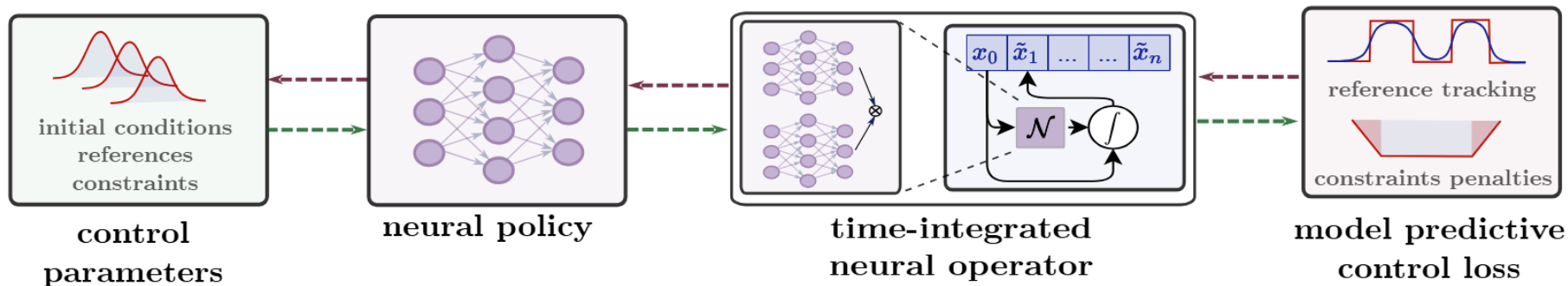
Output: Trained explicit policy π_{θ^*}

```

1 while not converged do
2     Sample mini-batch  $\{(x_0^{(i)}, \xi^{(i)})\}_{i=1}^B \sim \mathcal{X}_0 \times \mathcal{D}$ ;
3     for  $i \leftarrow 1$  to  $B$  do
4         for  $k \leftarrow 0$  to  $N - 1$  do
5              $u_k^{(i)} \leftarrow \pi_\theta(x_k^{(i)}; \xi_k^{(i)})$ ;
6              $x_{k+1}^{(i)} \leftarrow f(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)})$ ;
7              $\ell_k^{(i)} \leftarrow \ell(x_k^{(i)}, u_k^{(i)}; \xi_k^{(i)}) + Q_x p_x(g(x_k^{(i)}; \xi_k^{(i)})) + Q_u p_u(h(u_k^{(i)}; \xi_k^{(i)}))$ ;
8              $\mathcal{L}^{(i)} \leftarrow \frac{1}{N} \sum_{k=0}^{N-1} \ell_k^{(i)} + \ell_N(x_N^{(i)}; \xi_N^{(i)})$ ;
9          $\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^B \mathcal{L}^{(i)}$ ; // batch DPC loss
10        Compute  $\nabla_\theta \mathcal{L}$  (e.g., BPTT or adjoint sensitivity using (8.3)–(8.4));
11         $\theta \leftarrow \mathbb{O}(\theta, \nabla_\theta \mathcal{L})$ ; // e.g., Adam step
12 return  $\pi_\theta$ ;
```

Ok, but what if the system is PDE?

Learning to Control PDEs with DPC and Neural Operators



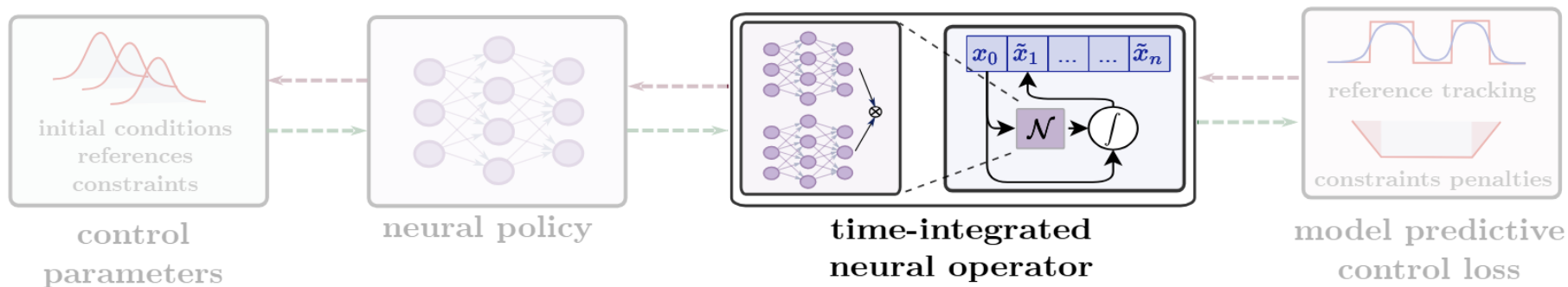
$$\min_{a(\cdot, \cdot)} J[u, a] = \int_0^T \int_{\Omega} \ell(u(t, \mathbf{x}), a(t, \mathbf{x}), \boldsymbol{\xi}(t)) d\mathbf{x} dt + \int_{\Omega} \ell_T(u(T, \mathbf{x})) d\mathbf{x},$$

$$\text{s.t.} \quad \frac{\partial u}{\partial t} = \mathcal{F}(t, \mathbf{x}, u, \nabla u, \nabla^2 u, \dots, a),$$

$$h(u(\mathbf{x}, t), \boldsymbol{\xi}(t)) \leq 0, \quad g(a(\mathbf{x}, t), \boldsymbol{\xi}(t)) \leq 0,$$

$$u(\mathbf{x}, 0) = u_0(\mathbf{x}), \quad \mathcal{B}(u) = 0.$$

Time-Integrated Neural Operator (TI-DeepOnet)



TI-DeepOnet: PDE surrogate based on method-of-lines spatial discretization

$$\partial u / \partial t \approx \mathcal{G}_{\theta}(u^i, a^i)(\zeta) = \sum_{j=1}^p [b_j^u(u^i) \odot b_j^a(a^i)] t_j(\zeta)$$

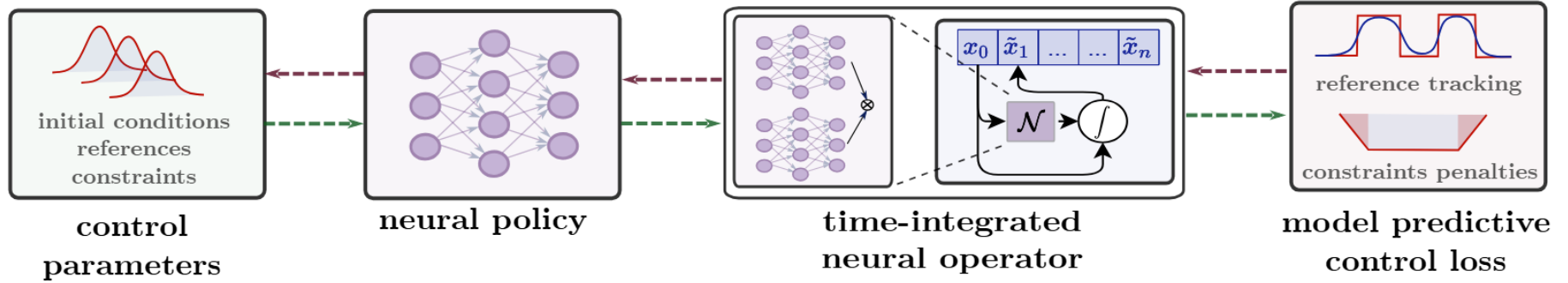
$\odot$ denotes element-wise multiplication

b_j^u is a state branch net encoding the state field

b_j^a is a control branch net encoding the control function

t_j is a trunk net encoding the solution at the collocation points ζ

Formulation of DPC with TI-DeepOnet



Discretize then optimize reformulation of the PDE control problem:

$$\min_{\mathbf{W}} \quad \mathbb{E}_{u_0 \sim \mathcal{P}_{u_0}, \xi \sim \mathcal{P}_{\xi}} \left[\sum_{k=0}^{N-1} \sum_{i=1}^{n_x} \ell(u_k^i, a_k^i, \xi_k) \Delta x + \sum_{i=1}^{n_x} \ell_N(u_N^i) \Delta x \right],$$

$$\text{s.t.} \quad \mathbf{u}_{k+1} = \text{ODESolve}(\mathcal{G}_{\theta}(t_k, \mathbf{u}_k, \mathbf{a}_k)),$$

$$\mathbf{a}_k = \pi_{\mathbf{W}}(\mathbf{u}_k, \xi_k), \quad h(\mathbf{u}_k, \xi_k) \leq 0, \quad g(\mathbf{a}_k, \xi_k) \leq 0,$$

Training Algorithms for TI-DeepOnet and DPC

Algorithm 1 Train TI-DON

- 1: Training datasets of initial condition, problem parameters ξ_k and control inputs from $\mathcal{P}_{u_0}$, $\mathcal{P}_\xi$ and $\mathcal{P}_a$ respectively.
 - 2: Neural operator, $\mathcal{G}_\theta : (u(t), f(t)) \mapsto \partial_t u$
 - 3: Time stepping, $u(t+1) = \text{RK4}(\mathcal{G}_\theta)$
 - 4: Mean square error loss, $\mathcal{L}_{\text{mse}}$
 - 5: Optimizer $\mathbb{O}$
 - 6: Learn, θ via optimizer $\mathbb{O}$ using gradient $\nabla_\theta \mathcal{L}_{\text{mse}}$
 - 7: **return** trained neural operator, $\mathcal{G}_\theta$
-

Algorithm 2 DPC offline training

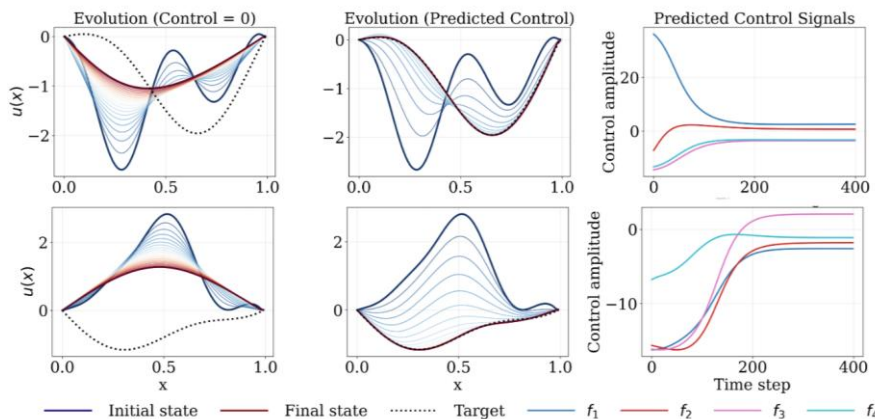
- 1: Training datasets of initial condition and problem parameters ξ_k from $\mathcal{P}_{u_0}$ and $\mathcal{P}_\xi$ respectively.
 - 2: Pretrained neural operator, $\mathcal{G}_\theta$
 - 3: Time stepping, $u(t+1) = \text{RK4}(\mathcal{G}_\theta)$
 - 4: DPC loss. $\mathcal{L}_{\text{DPC}}$ (6)
 - 5: Optimizer $\mathbb{O}$
 - 6: Learn, $\mathbf{W}$ via optimizer $\mathbb{O}$ using gradient $\nabla_{\mathbf{W}} \mathcal{L}_{\text{DPC}}$
 - 7: **return** optimized policy $\pi_{\mathbf{W}}(\mathbf{u}_k, \xi_k)$
-

JAX implementation: https://github.com/Centrum-IntelliPhysics/PDEControl_DPC

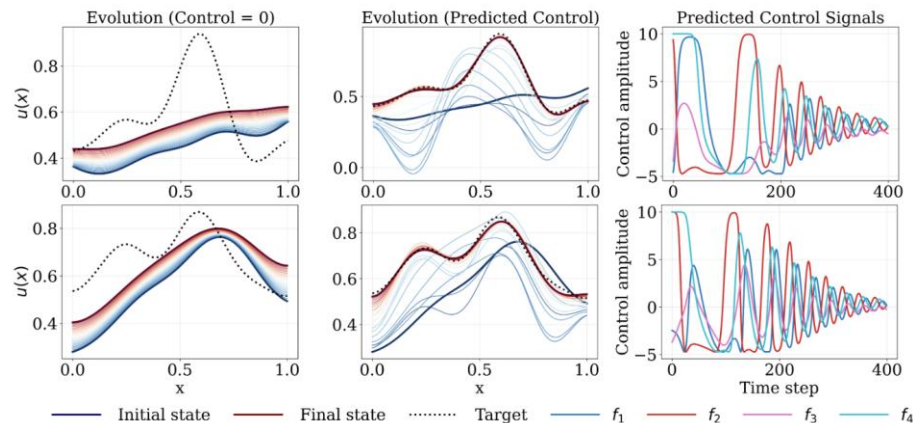
Learning to Control PDEs with DPC and Neural Operators

Algorithm	Metric	HE TT (IV-B)	BE (IV-C)	RDE (IV-D)
NMPC	MSE	$(4.2 \pm 6.7) \times 10^{-3}$	$(3.1 \pm 2.9) \times 10^{+6}$	$(1.8 \pm 1.6) \times 10^{-3}$
	Time/step (s)	4.3 ± 1.5	3.9 ± 13	7.9 ± 1.6
TIDON-DPC	MSE	$(7.7 \pm 20) \times 10^{-3}$	$(4.4 \pm 9.3) \times 10^{+7}$	$(1.4 \pm 1.4) \times 10^{-3}$
	Time/step (s)	$(2.5 \pm 2.2) \times 10^{-4}$	$(8.8 \pm 1.3) \times 10^{-5}$	$(2.4 \pm 2.4) \times 10^{-4}$

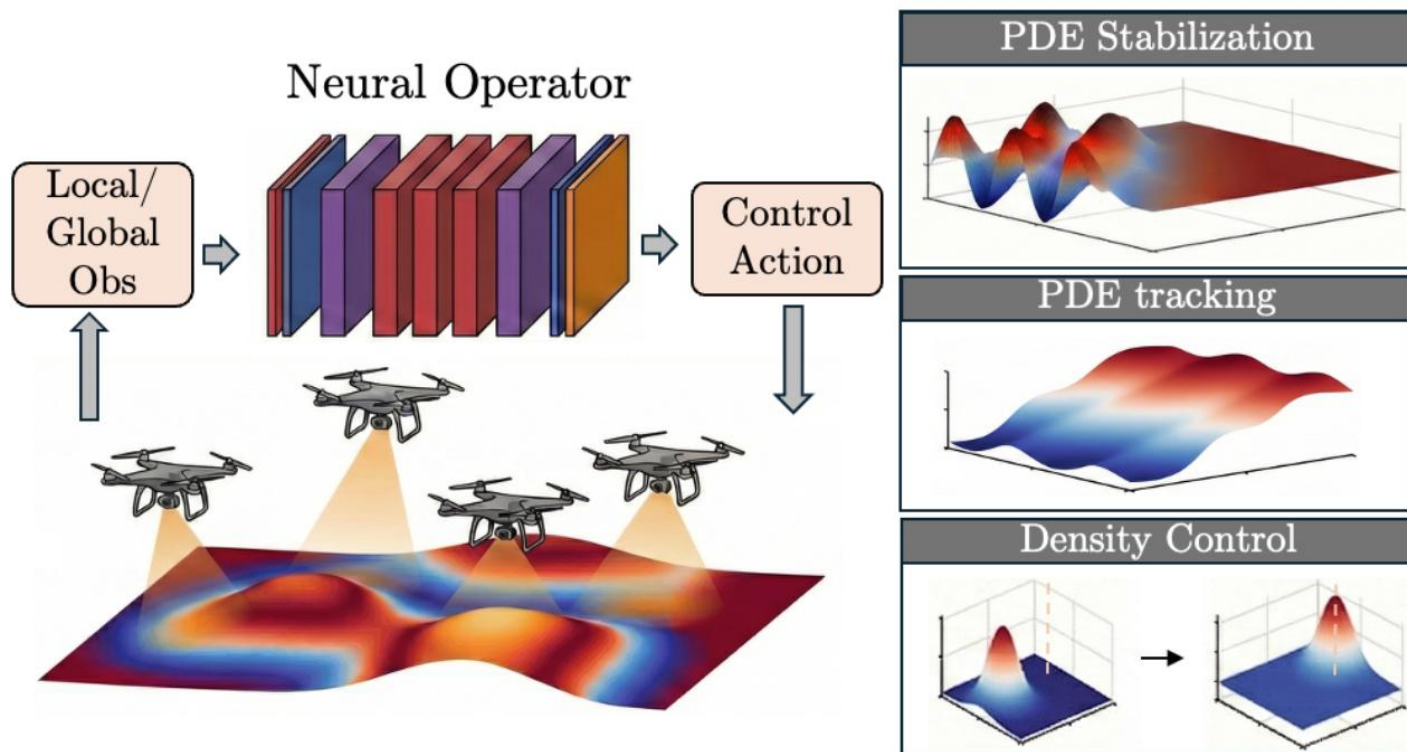
Heat equation tracking control



Fisher-KPP population density control



Cardinality-Invariant Neural Operator Policies for Multi-Agent PDE Control



Website with code.

<https://solaris-jhu.github.io/CINOC-website/>

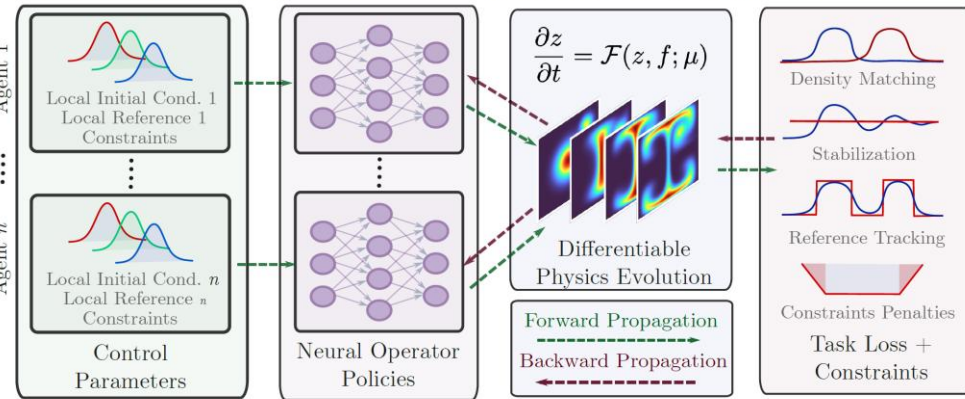
Multi-agent PDE Control Problem

$$\begin{aligned}
 \min_{\boldsymbol{\theta}} \quad & \mathcal{J} = \mathbb{E}_{\boldsymbol{\psi} \sim \mathcal{D}_{\text{prob}}} \left[\int_0^T \ell(z(\mathbf{x}, t), \mathbf{u}^M(t), \boldsymbol{\psi}), dt \right] \longrightarrow \text{Parametric control functional} \\
 \text{s.t.} \quad & \frac{\partial z}{\partial t} = \mathcal{F} \left(z, \sum_{i=1}^M \mathcal{B}(\mathbf{x}, u_i(t), \boldsymbol{\xi}_i(t)) \right), \longrightarrow \text{Time-dependent PDE operator} \\
 & \dot{\boldsymbol{\xi}}_i(t) = \mathcal{K}(\boldsymbol{\xi}_i(t), \mathbf{v}_i(t)), \quad i = 1, \dots, M, \longrightarrow \text{Actuator ODE dynamics} \\
 & z_i^{\text{patch}}(t) = \mathcal{Obs}(z(\mathbf{x}, t), \boldsymbol{\xi}_i(t), \boldsymbol{\psi}), \longrightarrow \text{Sensor operator} \\
 & [u_i(t), \mathbf{v}_i]^\top = \mathcal{G}_{\boldsymbol{\theta}}(z_i^{\text{patch}}, \boldsymbol{\xi}_i), \longrightarrow \text{Shared neural operator policy} \\
 & \mathbf{c}(z(\mathbf{x}, t), \mathbf{u}^M(t), \boldsymbol{\psi}) \leq 0. \longrightarrow \text{Constraints}
 \end{aligned}$$

PDE Control as Self-Supervised Neural Operator Learning

We learn the multi-agent policy operator using differentiable predictive control algorithm with differentiable PDE solvers.

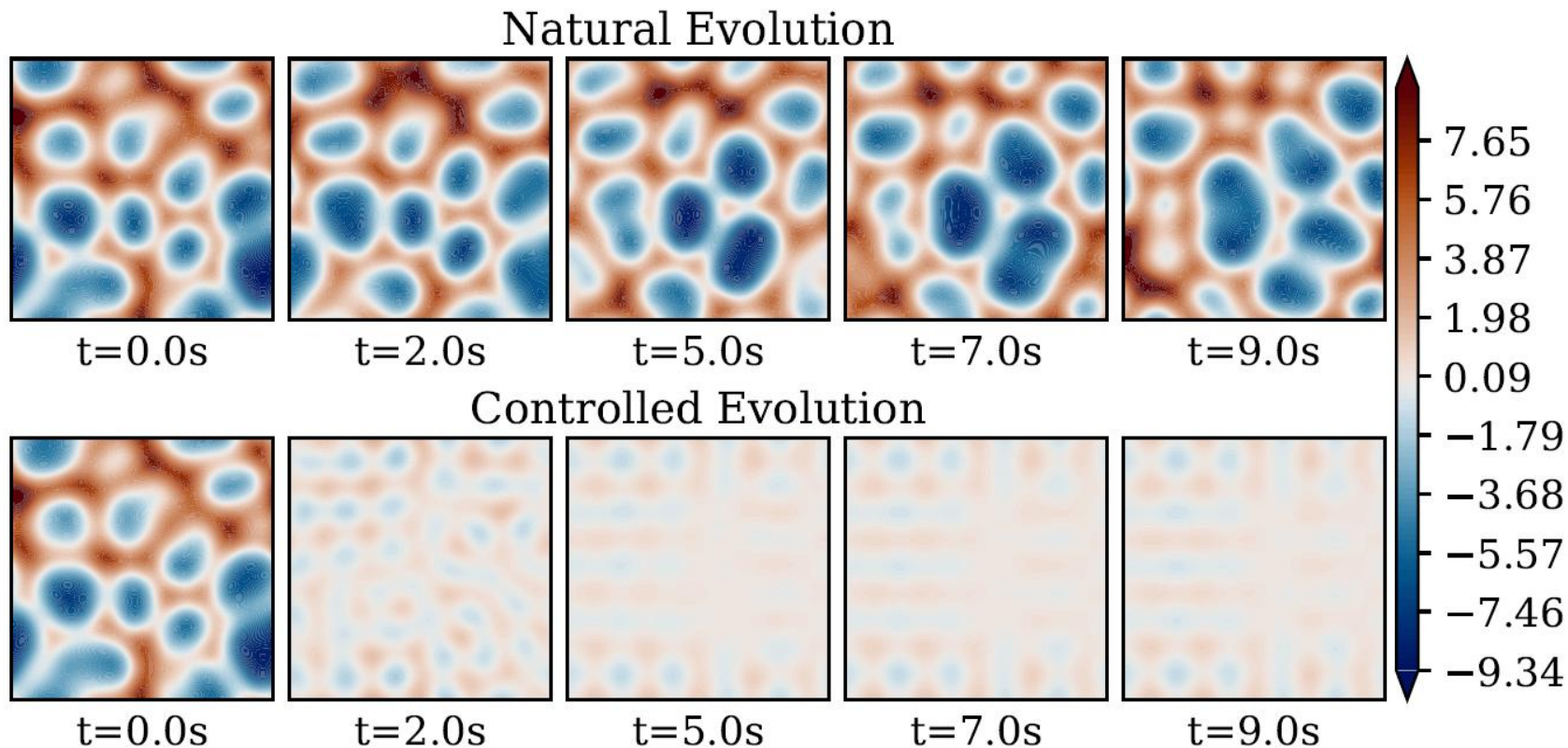
Neural Operator Policies for Cardinality Invariant PDE Control



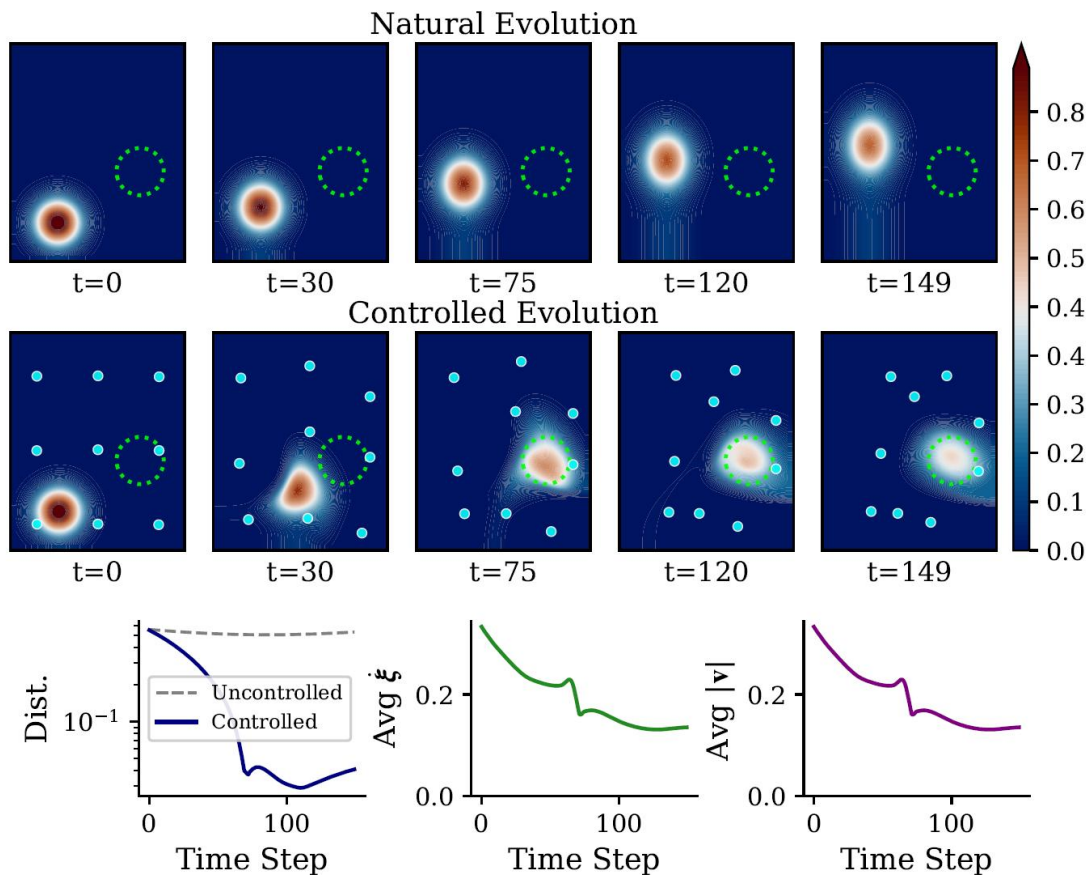
Algorithm 1 Self-Supervised Operator Policy Training

- 1: **Input:** distribution $\mathcal{D}_{\text{prob}}$, horizon T , number of agents M
- 2: **Initialize:** shared policy parameters θ
- 3: **while** not converged **do**
- 4: **{1. Data Sampling}**
- 5: Sample instances $\psi = (z_0, z_{\text{target}}, \dots) \sim \mathcal{D}_{\text{prob}}$
- 6: Initialize state z_0 , actuator positions $\{\xi_{i,t=0}\}_{i=1}^M$
- 7: **for** $t = 0$ to T **do**
- 8: **{2. Shared Policy Inference}**
- 9: **for** $i = 1$ to M **do**
- 10: $z_i^{\text{patch}} \leftarrow \text{Obs}(z_t, \xi_{i,t}, \psi)$
- 11: $[u_i(t), v_i]^\top \leftarrow \mathcal{G}_\theta(z_i^{\text{patch}}, \xi_{i,t})$
- 12: **end for**
- 13: **{3. Differentiable Physics Step}**
- 14: $z_{t+1}, \{\xi_{t+1}\} \leftarrow \Psi(z_t, \{\xi_t\}, u_t^M)$
- 15: **{4. Loss}**
- 16: $\mathcal{J} \leftarrow \mathcal{J} + \ell(z_{t+1}, u_t^M, \psi)$
- 17: **end for**
- 18: **Update:** $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{J}$ **{Backprop via Physics}**
- 19: **end while**

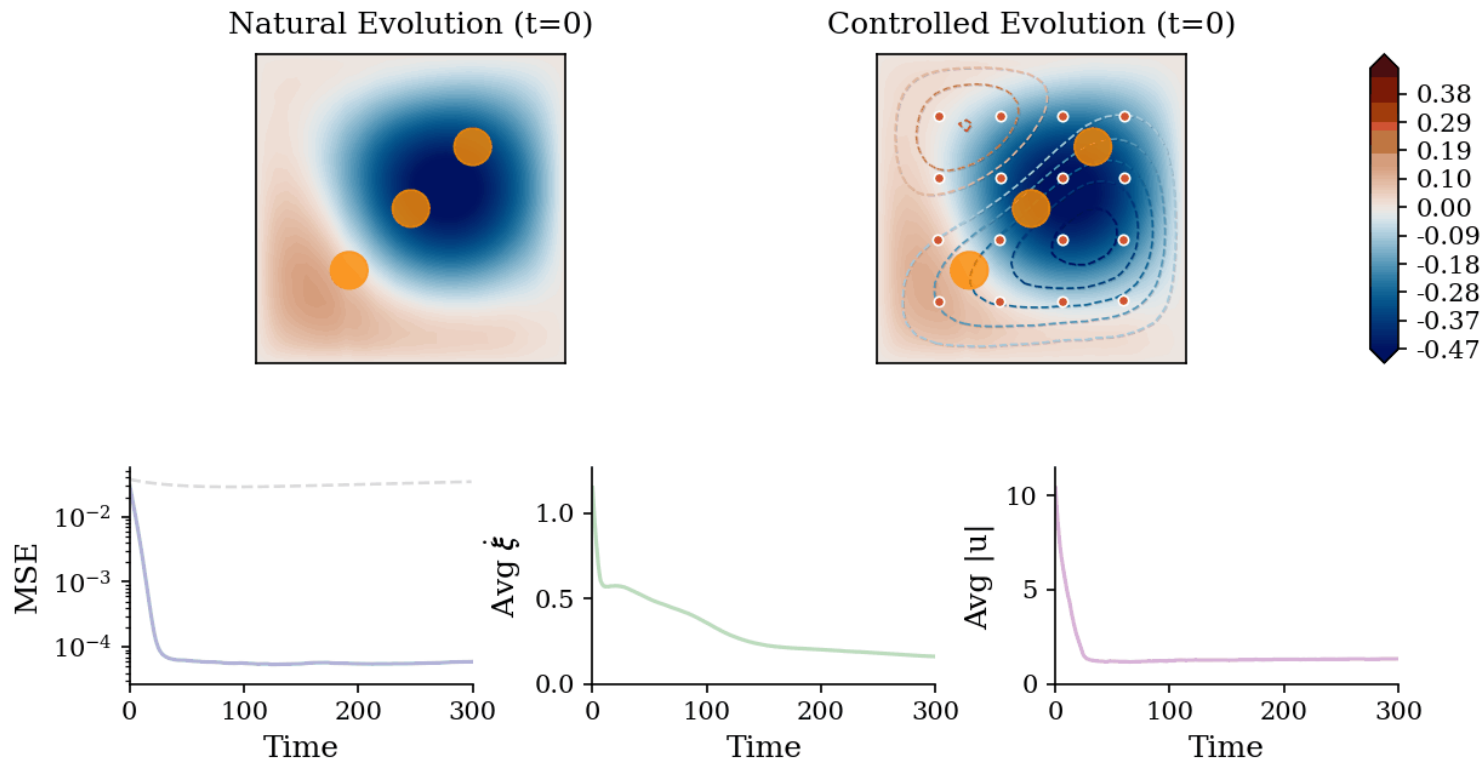
Empirical Results: Stabilization of 2D Kuramoto-Sivashinsky Chaos



Empirical Results: Density Transport in Advection-Diffusion

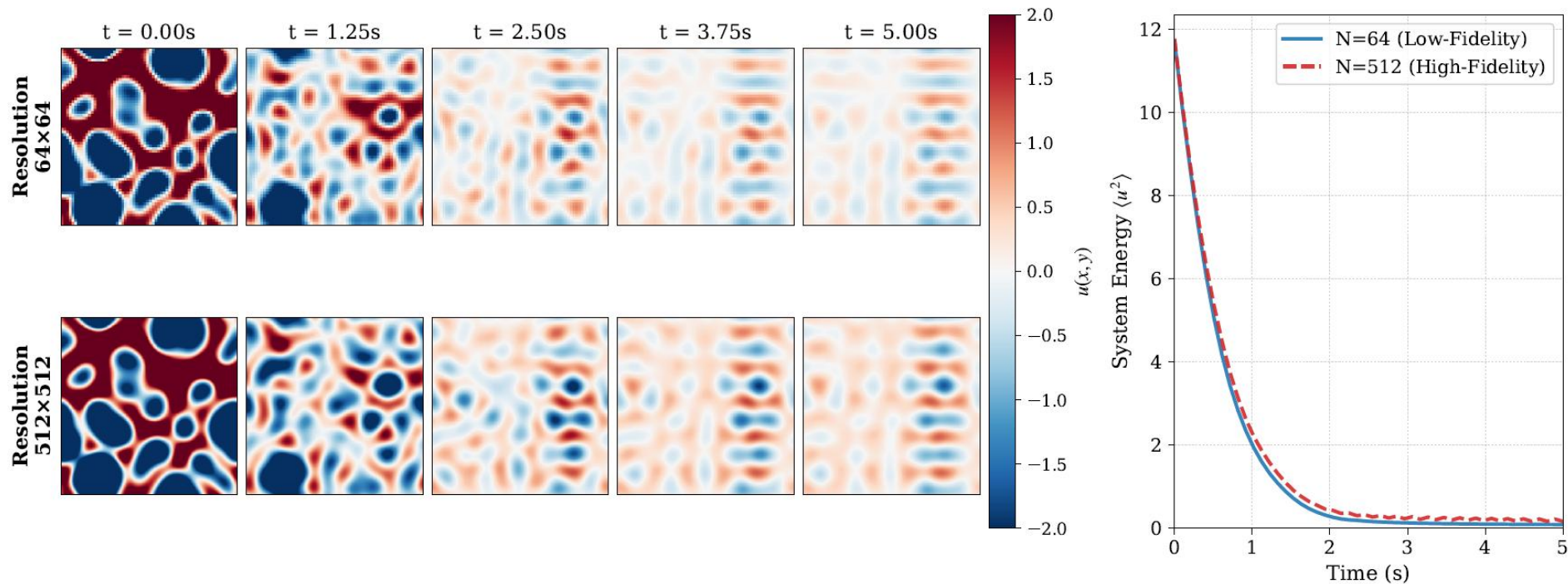


Empirical Results: Multi-agent Heat Control with Spatial Constraints



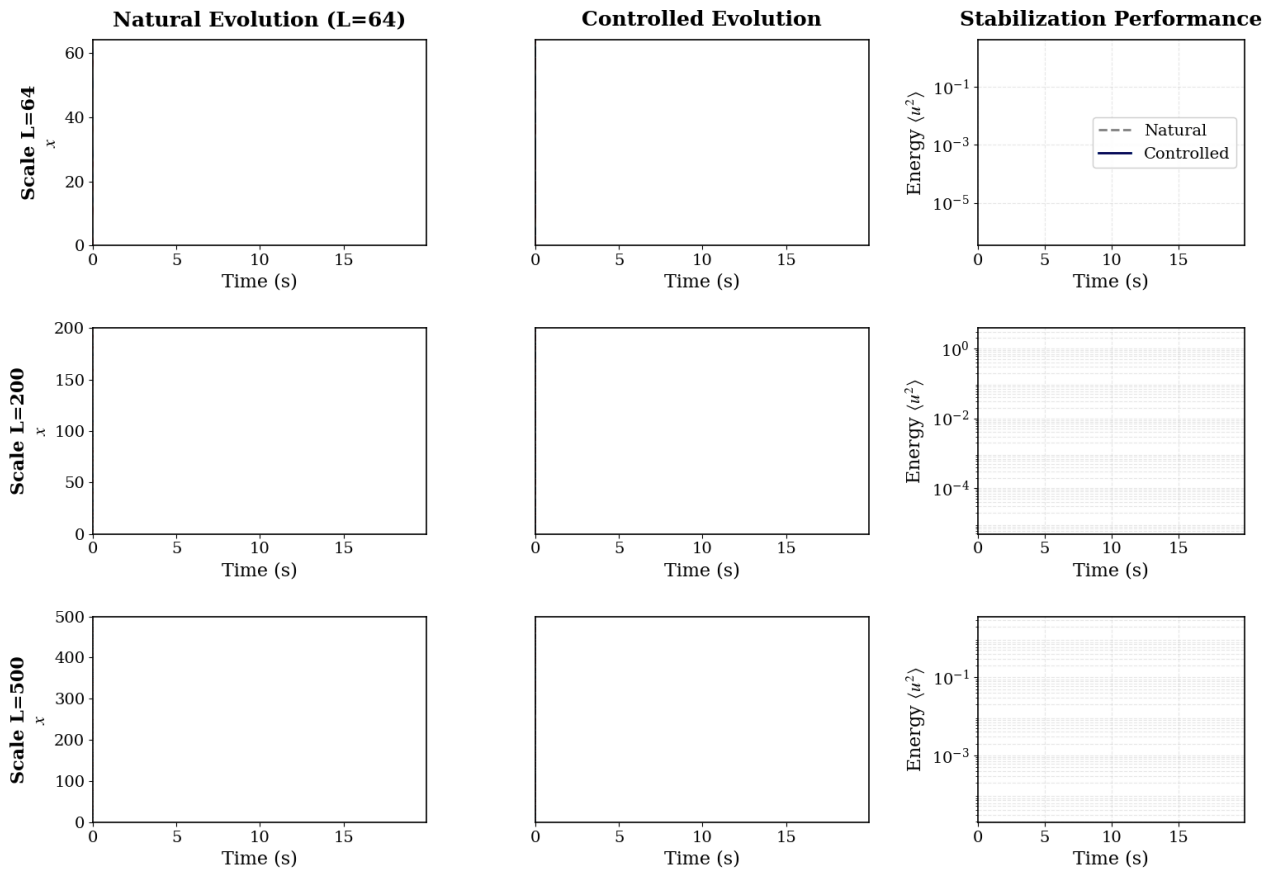
Empirical Results: Solver Fidelity Robustness

The policy, trained on a coarse discretization of 64×64 (Top), is deployed zero-shot on a high-fidelity solver with 512×512 grid points (Bottom).

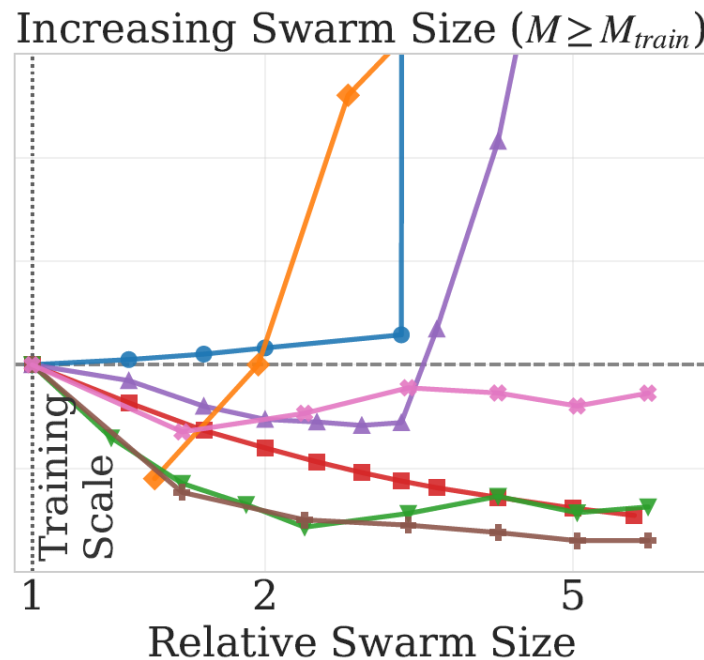
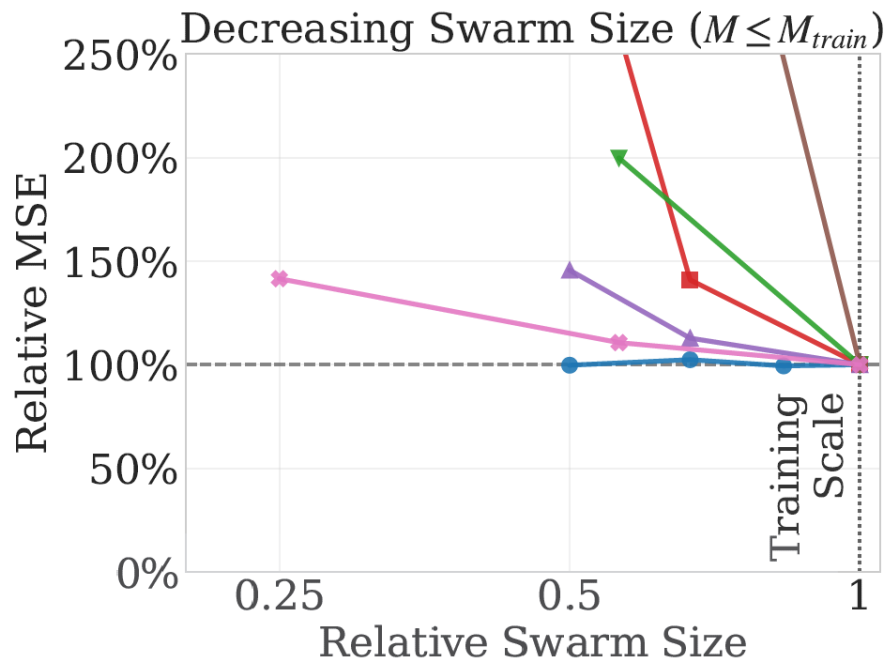
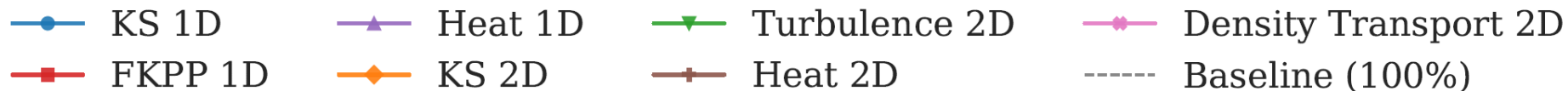


Empirical Results: Solver Fidelity Robustness

Stabilizing 1D Kuramoto-Sivashinsky across actuator scales with single neural operator policy.



Cardinality Invariance Across Diverse PDE Benchmarks



Mean-Field Gradient Consistency

We train a **shared neural operator policy** on a *finite swarm* of size M , but **deploy it zero-shot** on *different swarm sizes*. Key questions:

- Does finite- M training optimize the *true continuous control problem*?
- Why does the policy transfer across cardinalities without knowing M ?

We observe that training on a finite swarm asymptotically optimizes the *continuous PDE control operator*, not a particular agent configuration. Hence cardinality becomes a *resolution parameter*.

As the number of agents grows, discrete actuation converges to a **continuous actuator density**.

$$\sum_{i=1}^M B(x, \xi_i) u_i \longrightarrow \mathcal{B}_{\infty}(z) \quad (\text{mean-field forcing operator})$$

Our primary theoretical result establishes that the gradients computed during our discrete multi-agent training are consistent estimators of the gradients for this continuous limit.

Mean-Field Gradient Consistency

Theorem 5.1 (Consistency of Discrete Policy Gradients (Informal)). *As the number of agents $M \rightarrow \infty$, the discrete policy gradient $\nabla_{\theta} \mathcal{J}_M$ converges to the mean-field gradient $\mathcal{D}_{\theta} \mathcal{J}_{\infty}$.*

Proof sketch: Proof relies on the discrete physics uniformly approximates the mean-field physics.

- 1. Uniform bounds:** PDE states and adjoints remain uniformly bounded in suitable function space regardless of M , preventing blow-ups as agents are added.
- 2. Compactness and convergence:** Using the Aubin-Lions lemma, we extract strongly convergent subsequences of the state trajectories, showing that the discrete dynamics converge to a unique mean-field limit.
- 3. Gradient Identification:** We decompose the gradient error into a state-dependent term and a measure-dependent term. We show both vanish in the limit, proving that optimizing the swarm is asymptotically equivalent to optimizing the continuous operator.

Acknowledgments



**Pietro
Zanotta**



**Dibakar Roy
Sharkar**



**Honghui
Zheng**



**Somdatta
Goswami**



Aaron Tuor



**Draguna
Vrabie**

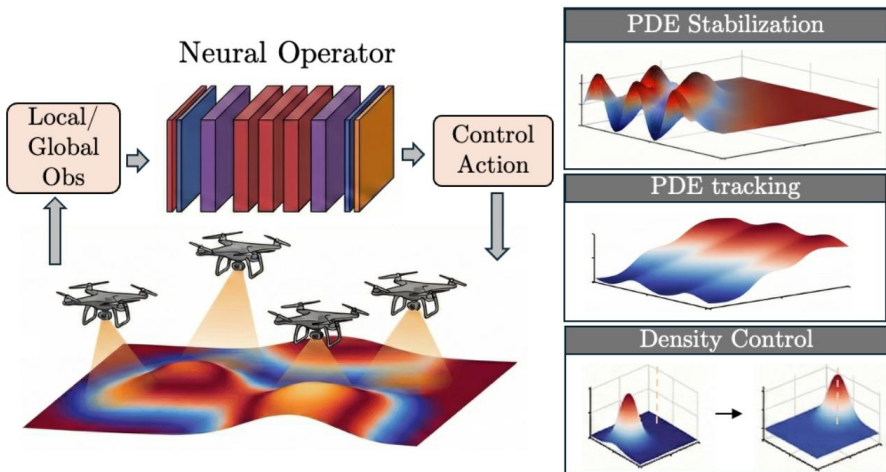


Ján Drgoňa



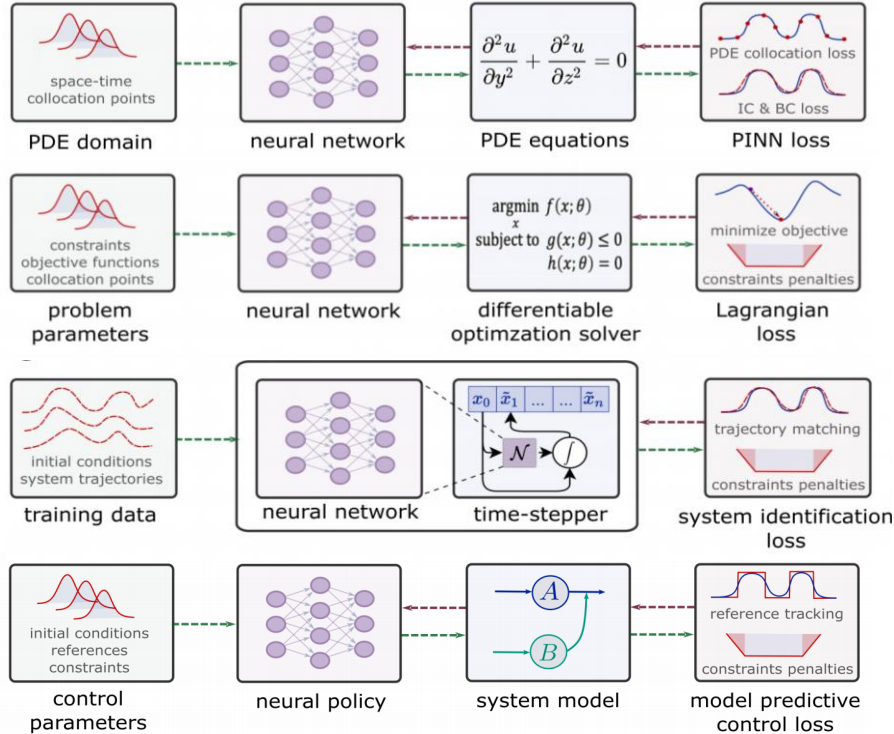
Summary

- **Differentiable Predictive Control (DPC) with Neural Operators**
 - Enables scalable policy optimization for parabolic PDEs
 - Works both with differentiable PDE solves and neural operator surrogates
 - Cardinality invariant scalability



- **Challenges**
 - Control of hyperbolic PDEs
 - Gradients ill conditioning
 - Mixed-integer decision domains
 - Offline pre-training vs online adaptation
- **Code**
 - github.com/SOLARIS-JHU/CINOC
 - github.com/Centrum-IntelliPhysics/PDEControl_DPC
 - github.com/pnnl/neuromancer
- **Hiring!**
 - Interested? Shoot me an email: jdrгона1@jh.edu

NeuroMANCER Scientific Machine Learning Library



Open-source library in PyTorch

- Physics-informed Neural Networks
- Learning to optimize
- Neural differential equations
- Learning to control



github.com/pnnl/neuromancer

NeuroMANCER Team



Ján Drgoňa



Aaron Tuor



James Koch



**Madelyn
Shapiro**



**Rahul
Birmiwal**



**Bruno
Jacob**



**Draguna
Vrabie**



JOHNS HOPKINS
UNIVERSITY



U.S. DEPARTMENT OF
ENERGY



Pacific Northwest
NATIONAL LABORATORY

DPC vs Model-based Reinforcement Learning

DPC is closely related to **MBRL** in that both leverage **model of dynamics**, but DPC utilizes **differentiable** closed-loop models and cost functions, allowing direct policy gradients **without requiring a learned critic**.

Empirical risk minimization problem:

$$\min_{\theta} \mathbb{E}_{\mathbf{x}_0, \xi_k} \left(\sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k, \xi_k) + p_N(\mathbf{x}_N) \right)$$

$$\text{s.t. } \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \xi_k), \quad k \in \mathbb{N}_0^{N-1}$$

$$\mathbf{u}_k = \pi_{\theta}(\mathbf{x}_k, \xi_k)$$

$$h(\mathbf{x}_k, \xi_k) \leq 0$$

$$g(\mathbf{u}_k, \xi_k) \leq 0$$

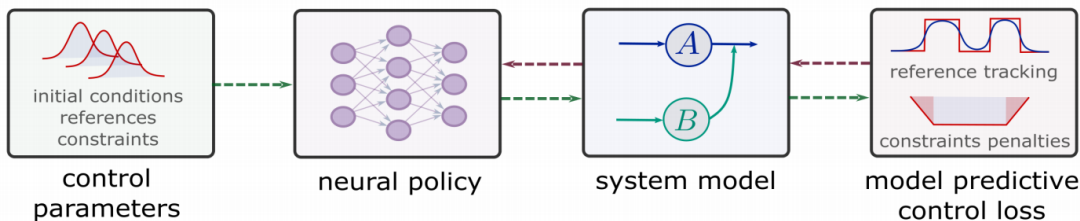
Critic parametrized by differentiable MPC loss function:

$$\mathcal{L}_{\text{DPC}} = \frac{1}{mN} \sum_{i=1}^m \left(\sum_{k=0}^{N-1} (\ell(\mathbf{x}_k^i, \mathbf{u}_k^i, \xi_k^i) + \right.$$

$$\left. p_x(h(\mathbf{x}_k^i, \xi_k^i)) + p_u(g(\mathbf{u}_k^i, \xi_k^i)) \right) + p_N(\mathbf{x}_N^i))$$

Policy gradient via automatic differentiation:

$$\nabla_{\theta} \mathcal{L}_{\text{DPC}} = \left(\frac{\partial \ell}{\partial \mathbf{x}} + \frac{\partial p_x}{\partial \mathbf{x}} + \frac{\partial p_N}{\partial \mathbf{x}} \right) \cdot \frac{\partial \mathbf{x}}{\partial \mathbf{u}} \cdot \frac{\partial \mathbf{u}}{\partial \theta} + \left(\frac{\partial \ell}{\partial \mathbf{u}} + \frac{\partial p_u}{\partial \mathbf{u}} \right) \cdot \frac{\partial \mathbf{u}}{\partial \theta}$$



$\ell(\cdot)$: stage cost

$p_x(\cdot)$, $p_u(\cdot)$: constraints penalties

$p_N(\cdot)$: terminal cost

$\frac{\partial \mathbf{x}}{\partial \mathbf{u}}$: differentiable dynamics

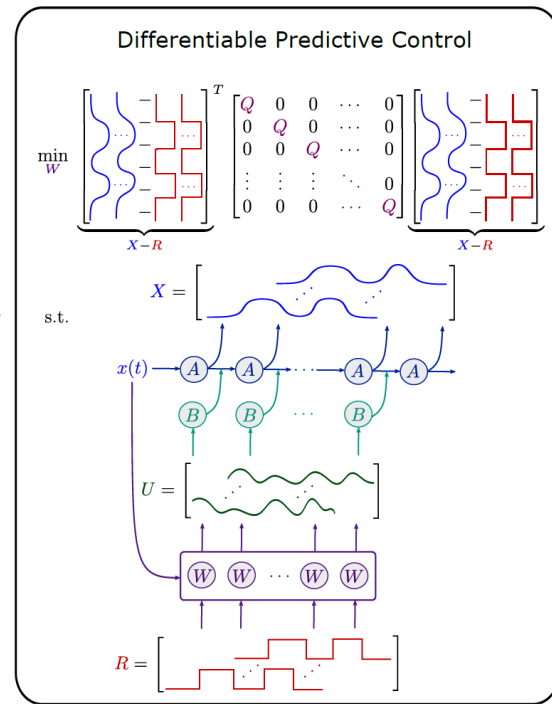
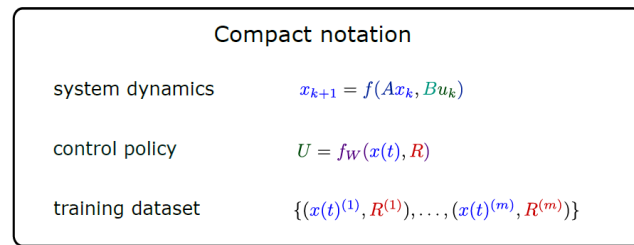
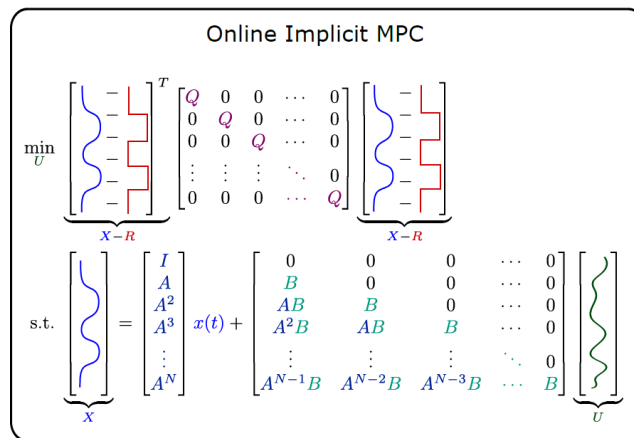
$\frac{\partial \mathbf{u}}{\partial \theta}$: differentiable policy

DPC vs Model Predictive Control

DPC is also closely related to MPC, but instead of single instance online optimization, the **DPC learns parametric explicit policy offline**, over distribution of parametric instances in batched setting.

There is a **structural equivalence** between *single shooting* formulation of MPC problem and the *unrolled closed-loop system* dynamics in DPC.

DPC is also related to **explicit MPC**, as it solves a **parametric optimal control problem** via gradient-based policy optimization.



structural
equivalence